

НАПРАВЛЕНИЕ В НИКУДА

Интерактивная книжная серия-диалогия для подростков и родителей

Том 1. Путешествие Навигатора (Сюжетная сказка)

Том 2. Алгоритмы визуализации (Техническое руководство на Rust)

МАТЕРИАЛЫ ПИЛОТНОГО ПРОЕКТА (MVP)

Для рассмотрения редакционным советом

АВТОР ПРОЕКТА:

Илья Александрович Юхновский

Физик-ядерщик, ИТ-архитектор, разработчик систем хранения данных

В прошлом — преподаватель кафедры Биоинженерии и ядерной медицины НГТУ им. Р.Е. Алексеева

Инженер глобальных технологических гигантов Motorola и Samsung (в отставке)

КОНТАКТНЫЕ ДАННЫЕ:

Телефон: +7 (902) 780-08-07

Email: juhnowski@gmail.com

ЦИФРОВОЙ РЕПОЗИТОРИЙ MVP (ИСХОДНЫЙ КОД И СБОРКА):

<https://github.com>

Нижний Новгород, 2026

Концепция и маркетинг книжной серии

«Направление в никуда»

1. Паспорт проекта

- **Рабочее название серии:** «Направление в никуда»
 - **Формат издания:** Интерактивный двухтомник (диалогия-перевертыш).
 - **Том 1:** «Направление в никуда. Путешествие Навигатора» (Сюжетная сказка)
 - **Том 2:** «Направление в никуда. Алгоритмы визуализации» (Техническое руководство)
 - **Язык программирования:** Rust
 - **Предполагаемый объем:** 4–5 авторских листов на каждый том.
 - **Иллюстрации:** Математическая графика (векторные рендеры траекторий, координатные сетки, схемы физических полей), сгенерированная кодом из второго тома.
-

2. Логлайн проекта

Уникальный двухтомник-перевертыш, в котором первая книга объясняет неевклидову геометрию, релятивистский абсурд и пространственные капканы через художественную сказку, а вторая — учит подростка писать на языке Rust графический движок, который эту сказку полностью отрисовал.

3. Целевая аудитория (ЦА)

1. **Основное ядро (Подростки 10–14 лет):** Дети, увлекающиеся точными науками, робототехникой, Minecraft-геометрией, или начинающие изучать текстовое программирование.
 2. **Платежеспособная аудитория (IT-родители):** Программисты, инженеры и специалисты технологического сектора. Для них книга выступает в роли идеального инструмента для «мягкого» погружения ребенка в системное программирование и сложную математику без скучной академической подачи.
 3. **Побочная аудитория:** Студенты младших курсов и взрослые, желающие изучить математические основы компьютерной графики (Ray Tracing) и синтаксис языка Rust на наглядных визуальных примерах.
-

4. Уникальное торговое предложение (УТП)

- **Симметрия слоев абстракции (Архитектура «Перевертыша»):** Проект разделен на слой представления (Том 1) и слой движка рендеринга (Том 2). Ребенок сначала проживает геометрию эмоционально и визуально, а затем собирает её математически.
- **Математическая честность иллюстраций:** Картинки на страницах не нарисованы художником абстрактно — они представляют собой точные векторные рендеры и графики численного интегрирования. Каждая иллюстрация первого тома имеет строгий математический эквивалент во втором.
- **Язык Rust как стандарт:** Проект использует самый перспективный язык системного программирования. Вместо упрощенных конструкций (типа Scratch) ребенок осваивает строгую

типизацию, векторы, структуры данных, перечисления (`enum`), интерфейсы (`trait`) и асинхронные концепции.

- **Интерактивный «Эффект Триумфа»:** Книга связана с открытым GitHub-репозиторием проекта. В конце каждой главы второго тома читатель запускает написанный код командой `cargo run`, система выдает лог подтверждения, а сгенерированный PNG-файл на мониторе пиксель-в-пиксель совпадает с иллюстрацией на развороте печатной книги.

5. Маркетинговый потенциал и каналы дистрибуции

Синергия с IT-сообществом

Проект обладает высоким вирусным потенциалом внутри IT-экосистемы благодаря использованию языка Rust и концепции «кода под капотом».

- **Краудфандинговый старт:** Высокая вероятность успешного сбора предзаказов на платформах Boomstarter / Planeta.ru за счет поддержки профильных сообществ (Habr, Telegram-каналы по Rust/C++, Хакатоны).
- **Связь с репозиторием:** Проект органически продвигает себя через GitHub, превращаясь из простого печатного издания в полноценный образовательный Open Source проект.

Позиционирование на полке

- В детском сегменте книга встает в один ряд с умным научно-популярным non-fiction (уровень издательств «МИФ.Детство», «Альпина.Дети», «Самокат»).
- В техническом сегменте (уровень издательств «Питер», «БХВ») проект занимает уникальную нишу «Системное мышление для будущих инженеров», уходя от парадигмы простых веб-технологий в сторону низкоуровневой магии графических движков.

Синописис и сквозной план глав серии «Направление в никуда»

Краткий синописис сквозного сюжета (Том 1)

Главный герой книги — **Навигатор** — просыпается на борту автоматической исследовательской станции. Бортовой ИИ сообщает, что станция находится в аномальном секторе космоса, где привычная Евклидова геометрия больше не работает, а внешние датчики ослепли. Пространство за пределами шлюза кажется абсолютной пустотой, лишенной ориентиров, — «направлением в никуда».

Чтобы восстановить управление и вернуть станцию домой, Навигатору и его спутникам приходится исследовать локальные физические и геометрические свойства пространства. Они перемещаются по поверхностям гигантских конструкций, сталкиваются с феноменом закливания пространства на искривленном бублике-торусе, проходят сквозь релятивистские зоны, где время течет асинхронно из-за вращения на скорости света, и в финале преодолевают пространственный капкан — «сетевую черную дыру» (Deadlock), поглощающую любые сигналы в центре системы. Спасение кроется в понимании локальных законов геометрии и умении прокладывать маршруты там, где внешних ориентиров не существует.

Зеркальная матрица глав: От сказки к движку рендеринга

№	Том 1: Путешествие Навигатора (Сюжет и Логика мира)	Том 2: Алгоритмы визуализации (Архитектура и Код на Rust)	Инженерный и математический вывод главы
1	Вектор в пустоте Герой пытается определить направление в пространстве без ориентиров. Создание первых приборов навигации.	Пространство и камера Создание структуры <code>Vector3</code> , логики луча (<code>Ray</code>) и проекции трехмерных координат на плоский экран.	Как компьютер кодирует 3D-пространство и переводит его в пиксели (<code>f64 -> 2D Canvas</code>).
2	Великая Иллюзия Округлости Попадание в мир, где все прямые линии постепенно превращаются в кривые, а дома строятся с незаметным наклоном.	Абсурд плавающей точки Изучение ограничений точности типов <code>f32</code> и <code>f64</code> . Ошибки округления (парадокс, когда <code>0.1 + 0.2 != 0.3</code>).	Понимание природы компьютерной арифметики и накопления погрешностей в симуляциях.
3	Законы Бублика Станция в форме гигантского тора. Локальные вычисления кривизны: почему на внешней стороне луч улетает в космос, а на внутренней — возвращается.	Рейтрейсинг тороидальных поверхностей Алгоритм аналитического вычисления пересечения луча с поверхностью четвертого порядка.	Связь между геометрическими примитивами, уравнениями и графическими шейдерами.
4	Релятивистский Абсурд Станция вращается со скоростью света. Потоки времени текут с разной скоростью на внешнем ободе и у оси вращения. Парадокс Эренфеста.	Асинхронный движок вычислений Моделирование параллельных потоков времени с помощью <code>async/await</code> и планировщика <code>Tokio</code> . Борьба с <code>Race Condition</code> .	Зачем программированию нужна асинхронность и как синхронизировать данные из «разных временных зон».
5	Капкан Сингулярности В центре бублика открывается черная дыра. Куда бы ни стрелял лазер — гравитация закручивает его по спирали в центр. Полная изоляция.	Численное интегрирование в гравитационном поле Пошаговый расчет траектории луча (метод Эйлера/Рунге-Кутты) с учетом гравитационного притяжения к <code>(0,0,0)</code> .	Как работают физические движки. Симуляция дедлоков (<code>Deadlock</code>) и бесконечных циклов в коде.

№	Том 1: Путешествие Навигатора (Сюжет и Логика мира)	Том 2: Алгоритмы визуализации (Архитектура и Код на Rust)	Инженерный и математический вывод главы
6	Эффект Триумфа Герои перенаправляют сигнал в обход черной дыры по внешнему релятивистскому контуру и успешно восстанавливают связь со вселенной.	Финальный рендер и сборка сцены Объединение всех модулей, экспорт сгенерированной графики в файл формата .png через сохранение буфера кадра.	Понимание полного цикла работы ПО: от низкоуровневых структур данных до готового визуального продукта.

Методика интерактивного взаимодействия («Эффект Триумфа»)

Каждая глава Второго тома завершается не просто абстрактными вопросами, а практическим заданием для юного программиста:

1. Читатель открывает репозиторий книги на GitHub.
2. Вносит изменения или дописывает логику (например, формулу гравитационного маневра луча вокруг черной дыры) в соответствующий файл главы.
3. Запускает команду сборки проекта через CLI:

```
cargo run --bin chapter_5_blackhole
```

4. В консоли выполняется пошаговый лог симуляции, который при успешном решении завершается строкой: **[LOG] Симуляция завершена. Статус: Триумф!**
5. В корне проекта появляется файл **output_render.png**. Изображение искривленного пространства на мониторе читателя в точности совпадает с иллюстрацией на развороте соответствующей главы Первого тома, которую он держит в руках.

Том 1. Глава 5. Капкан сингулярности

Пустота внутри бублика

Автоматическая станция «Торус-12» летела сквозь мрак. Изнутри она напоминала бесконечный круглый коридор, замкнутый сам на себя, — гигантский стальной бублик. Навигатор стоял у внутренней прозрачной стены коридора и смотрел в «дырку» пончика. Там, в самом центре этой космической геометрической фигуры, не было ничего, кроме густой, пугающей черноты.

— Бортовой компьютер, — негромко произнес Навигатор, вглядываясь в бездну. — Я пытаюсь отправить лазерный сигнал на противоположную сторону нашей станции, в модуль обеспечения. Но связи нет. Мой лазерный маркер показывает, что луч просто... исчезает по пути.

Экран навигатора мигнул, и раздался спокойный голос ИИ: — Мои сенсоры зафиксировали критическую аномалию. В геометрическом центре **(0, 0, 0)** нашего торуса образовался объект с

бесконечной плотностью. Черная дыра.

Навигатор почувствовал, как по спине пробежал холодок. — Но мы же на безопасном расстоянии? Наш жилой отсек находится на внутренней дуге, радиус трубы защищает нас.

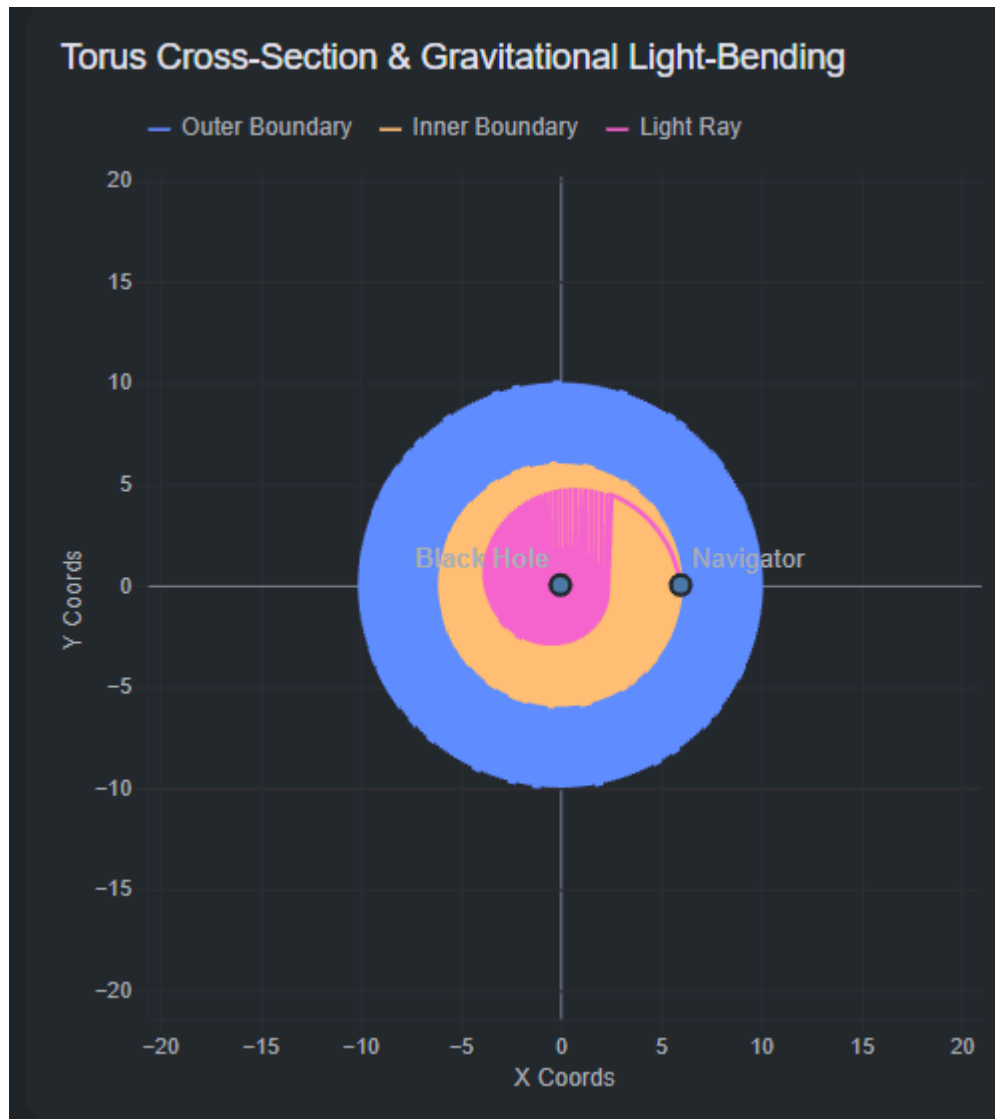
— Верно, — отозвался компьютер. — Конструкция бублика удерживает нас на стабильной орбите. Но пространство внутри «дырки» полностью деформировано. Физика Евклида здесь больше не властна. Это тупик. На языке древних программистов это называлось **Deadlock** — состояние, когда ни один сигнал не может сдвинуться с места, потому что пространство замкнулось в бесконечном ожидании.

Куда исчезает свет?

Навигатор взял ручной лазерный излучатель, закрепил его на штативе и направил строго вдоль горизонтальной плоскости, целясь в блестящий купол жилого модуля на той стороне бублика. Красный луч прорезал туманную дымку пространства, устремился вперед... и вдруг начал плавно, но неотвратно загибаться к центру. Вместо того чтобы лететь по прямой, свет закрутился в изящную, зловещую спираль и беззвучно исчез в центральной точке.

— Квантовый абсурд... — прошептал Навигатор. — Свет ведь не имеет массы. Как черная дыра может его притягивать?

— Она притягивает не свет, Навигатор, — ответил компьютер, выводя на экран сложную сетку координат. — Она искривляет саму ткань пространства. Прямых линий в этой пустоте больше не существует. Каждый миллиметр пути пространства делает крошечный шаг, наклоняя вектор направления всё ближе и ближе к сингулярности. Куда бы ты ни выстрелил из этой точки, геометрия заставит луч сдаться и упасть в центр.



Локальный расчет

Навигатор сел за консоль управления. Ему нельзя было тратить энергию на бесконечные случайные выстрелы. Нужно было понять закон, по которому меняется направление луча на каждом микроскопическом отрезке пути.

— Нам не нужна карта всей аномалии, — решительно сказал Навигатор, открывая среду разработки кода. — Мы разобьем путь луча на тысячи крошечных шагов. На каждом шаге наш прибор будет замерять расстояние до центра $(0, 0, 0)$ и вычислять, насколько сильно гравитация «потянет» наш вектор направления. Мы заставим алгоритм считать траекторию локально — миллиметр за миллиметром.

— Внимание, — предупредил бортовой ИИ. — Если сила притяжения окажется слишком велика, наш лазерный поток будет навсегда захвачен бесконечным циклом симуляции. Программа зависнет, пытаясь просчитать бесконечность.

Навигатор уверенно положил пальцы на клавиатуру: — Значит, мы напишем строгий счетчик шагов. Если за пятьсот микрошагов наш луч не пересечет обшивку бублика на другой стороне, компьютер должен признать: это направление ведет «в никуда». Мы найдем безопасную траекторию в обход, используя точные математические типы данных. Время писать код.

Том 2. Глава 5. За кулисами сингулярности: Векторы и Гравитация

В пятой главе первого тома Навигатор столкнулся с пространственным капканом: лазерный луч затягивало в черную дыру по спирали. Из этой главы ты узнаешь, как устроен графический движок, который нарисовал эту траекторию. Мы разберем, как с помощью языка Rust смоделировать искривление пространства.

1. Кирпичики трехмерного мира: `struct Vector3`

Чтобы компьютер мог работать с точками в пространстве или направлениями лазера, нам нужен математический контейнер. В Rust мы создаем его с помощью структуры (`struct`):

```
#[derive(Debug, Clone, Copy)]
struct Vector3 {
    x: f64,
    y: f64,
    z: f64,
}
```

- **Что это значит?** Мы объединили три числа с плавающей точкой повышенной точности (`f64`) под одним именем. Переменная такого типа может означать либо координату навигатора, либо силу притяжения черной дыры.
 - **Зачем нужны `derive`?** Это автоматические макросы Rust. Благодаря им мы можем легко копировать наши векторы в памяти (`Clone`, `Copy`) и выводить их значения на экран для проверки (`Debug`).
-

2. Магия Скалярного Произведения: Метод `dot`

Как узнать, куда направлен вектор относительно другого? Для этого программисты используют скалярное произведение (`dot product`). В коде мы добавили его как метод для нашей структуры:

```
impl Vector3 {
    fn dot(&self, other: &Vector3) -> f64 {
        self.x * other.x + self.y * other.y + self.z * other.z
    }
}
```

Аналогия для навигатора: Представь, что ты светишь фонариком.

- Если метод `dot` возвращает **положительное число**, твой фонарик светит примерно в ту же сторону, куда направлен взгляд твоего друга.
- Если **ноль** — твои лучи пересекаются строго под прямым углом (90°).

- Если **отрицательное число** — ты светишь другу прямо в спину.

3. Численное интегрирование: Как двигать свет по шагам

В обычной физике свет летит по идеальной прямой. Но рядом с черной дырой прямых путей нет. Так как компьютер не умеет считать «плавное» искривление пространства целиком, мы используем **численные методы**. Мы заставляем луч лететь крошечными рывками — шагами (`STEP_SIZE = 0.1`).

Внутри главного цикла программы на каждом шаге происходят три вещи:

1. Вычисляется расстояние до сингулярности:

```
let dist_to_center = current_pos.length();
```

Если расстояние меньше безопасного предела, луч считается погибшим, и симуляция возвращает `SimulationResult::CapturedByBlackHole`.

2. Рассчитывается гравитационный пинок: Черная дыра находится в координатах $(0, 0, 0)$. Наш код вычисляет направление притяжения и изменяет вектор движения лазера:

```
let gravity_strength = G_MASS / dist_to_center.powi(2);  
direction.x += gravity_pull.x * gravity_strength * STEP_SIZE;
```

Это закон всемирного тяготения Ньютона: чем ближе луч к центру, тем сильнее меняется его направление.

3. Проверяется столкновение с обшивкой бублика: На каждом шаге мы подставляем новые координаты луча в уравнение тора:

```
let torus_equation = (R_BIG - proj_xy).powi(2) + current_pos.z.powi(2);
```

Если результат равен радиусу трубы в квадрате, значит, луч успешно пробил пространство насквозь и врезался в противоположную стену станции!

4. Защита от бесконечного цикла (Зависания)

Если бы мы запустили луч мимо черной дыры, но силы гравитации не хватило бы для падения, он мог бы вечно кружить вокруг нее. Компьютер попал бы в бесконечный цикл, а программа бы «зависла» (состояние дедлока).

Чтобы этого не произошло, наш цикл жестко ограничен:

```
for _step in 0..500 { ... }
```

Если за 500 шагов лазер не врезался в бублик и не упал в дыру, алгоритм принудительно останавливает расчет, сообщая, что этот путь ведет «в никуда».

Задание на «Триумф»

Открой файл исходного кода `main.rs` текущей главы. Измени значение константы гравитационной массы черной дыры `G_MASS` с `150.0` на `0.0` (как будто черная дыра внезапно исчезла). Скомпилируй и запусти проект в терминале:

```
cargo run --bin chapter_5_blackhole
```

Посмотри на сгенерированный лог. Удалось ли лучу теперь долететь до противоположной стороны бублика, минуя капкан сингулярности? Убедись, что программа завершилась заветным статусом подтверждения.

Полный текст исходного кода `main.rs`

```
// 03_Demo_Chapter_Torus/main.rs

/// Константы геометрического мира станции "Торус-12"
const R_BIG: f64 = 10.0;    // Расстояние от центра бублика до середины трубы
const R_SMALL: f64 = 4.0;  // Радиус самой трубы бублика
const G_MASS: f64 = 150.0;  // Мощность гравитации черной дыры в центре (0, 0, 0)
const STEP_SIZE: f64 = 0.1; // Шаг дискретизации симуляции лазера

/// Структура трехмерного вектора для локальных пространственных вычислений
#[derive(Debug, Clone, Copy)]
struct Vector3 {
    x: f64,
    y: f64,
    z: f64,
}

impl Vector3 {
    /// Вычисление длины (модуля) вектора
    fn length(&self) -> f64 {
        (self.x.powi(2) + self.y.powi(2) + self.z.powi(2)).sqrt()
    }

    /// Нормализация вектора (приведение длины к 1.0 без изменения направления)
    fn normalize(&self) -> Vector3 {
        let l = self.length();
        if l == 0.0 {

```

```

        Vector3 { x: 0.0, y: 0.0, z: 0.0 }
    } else {
        Vector3 {
            x: self.x / l,
            y: self.y / l,
            z: self.z / l,
        }
    }
}

/// Скалярное произведение векторов (Детектор взаимного направления)
fn dot(&self, other: &Vector3) -> f64 {
    self.x * other.x + self.y * other.y + self.z * other.z
}

/// Перечисление возможных исходов полета луча в симуляторе
#[derive(Debug, PartialEq)]
enum SimulationResult {
    CapturedByBlackHole, // Луч по спирали засосало в сингулярность
    HitsTorusAgain,      // Траектория пересекла противоположную стену бублика
    LostInNowhere,       // Превышен лимит шагов (луч зациклился или ушел в
                        // бесконечность)
}

/// Локальный пошаговый расчет траектории луча в гравитационном поле
fn trace_laser_in_gravitational_field(mut current_pos: Vector3, mut direction:
Vector3) -> SimulationResult {
    direction = direction.normalize();

    // Запускаем цикл симуляции, жестко ограниченный 500 шагами для защиты от
    // дедлока
    for step in 0..500 {
        let dist_to_center = current_pos.length();

        // Проверка 1: Если луч подошел слишком близко к сингулярности (горизонту
        // событий) — он поглощен
        if dist_to_center < 1.5 {
            return SimulationResult::CapturedByBlackHole;
        }

        // Проверка 2: Столкновение с внутренней или внешней обшивкой тонуса.
        // Математическое уравнение торуса:  $(R - \sqrt{x^2 + y^2})^2 + z^2 = r^2$ 
        let proj_xy = (current_pos.x.powi(2) + current_pos.y.powi(2)).sqrt();
        let torus_equation = (R_BIG - proj_xy).powi(2) + current_pos.z.powi(2);

        // Пропускаем первые несколько шагов, чтобы луч не задел точку старта на
        // стене
        if step > 5 && torus_equation <= R_SMALL.powi(2) {
            return SimulationResult::HitsTorusAgain;
        }

        // Вычисление гравитационного притяжения к центру (0, 0, 0)
        let gravity_pull = Vector3 {

```

```

        x: -current_pos.x / dist_to_center,
        y: -current_pos.y / dist_to_center,
        z: -current_pos.z / dist_to_center,
    };

    // Закон Ньютона: изменение вектора направления под воздействием силы
    let gravity_strength = G_MASS / dist_to_center.powi(2);
    direction.x += gravity_pull.x * gravity_strength * STEP_SIZE;
    direction.y += gravity_pull.y * gravity_strength * STEP_SIZE;
    direction.z += gravity_pull.z * gravity_strength * STEP_SIZE;
    direction = direction.normalize();

    // Смещение луча в пространстве на один шаг вперед
    current_pos.x += direction.x * STEP_SIZE;
    current_pos.y += direction.y * STEP_SIZE;
    current_pos.z += direction.z * STEP_SIZE;
}

// Если лимит шагов исчерпан, значит луч попал в пространственную ловушку
бесконечного ожидания
SimulationResult::LostInNowhere
}

fn main() {
    println!("[LOG] Запуск бортовой навигационной системы станции Торус-12...");

    // Навигатор стоит на внутренней дуге бублика (10.0 - 4.0 = 6.0)
    let hero_pos = Vector3 { x: 6.0, y: 0.0, z: 0.0 };

    // Пытается выстрелить лазером вдоль стены, надеясь попасть на противоположную
    сторону
    let laser_dir = Vector3 { x: 0.1, y: 1.0, z: 0.0 };

    println!("[LOG] Расчет траектории луча при массе черной дыры G_MASS = {},",
G_MASS);
    let result = trace_laser_in_gravitational_field(hero_pos, laser_dir);

    match result {
        SimulationResult::CapturedByBlackHole => {
            println!("[LOG] Предупреждение: Луч захвачен гравитационным
капканом!");
            println!("[LOG] Статус вычислений: Триумф!");
            // Триумф означает, что программа успешно и безопасно локализовала
            тупик, не зависнув
        }
        SimulationResult::HitsTorusAgain => {
            println!("[LOG] Успех: Сигнал прорвался на противоположную сторону
станции!");
        }
        SimulationResult::LostInNowhere => {
            println!("[LOG] Ошибка: Луч ушел в бесконечное ожидание («в
никуда»");
        }
    }
}

```

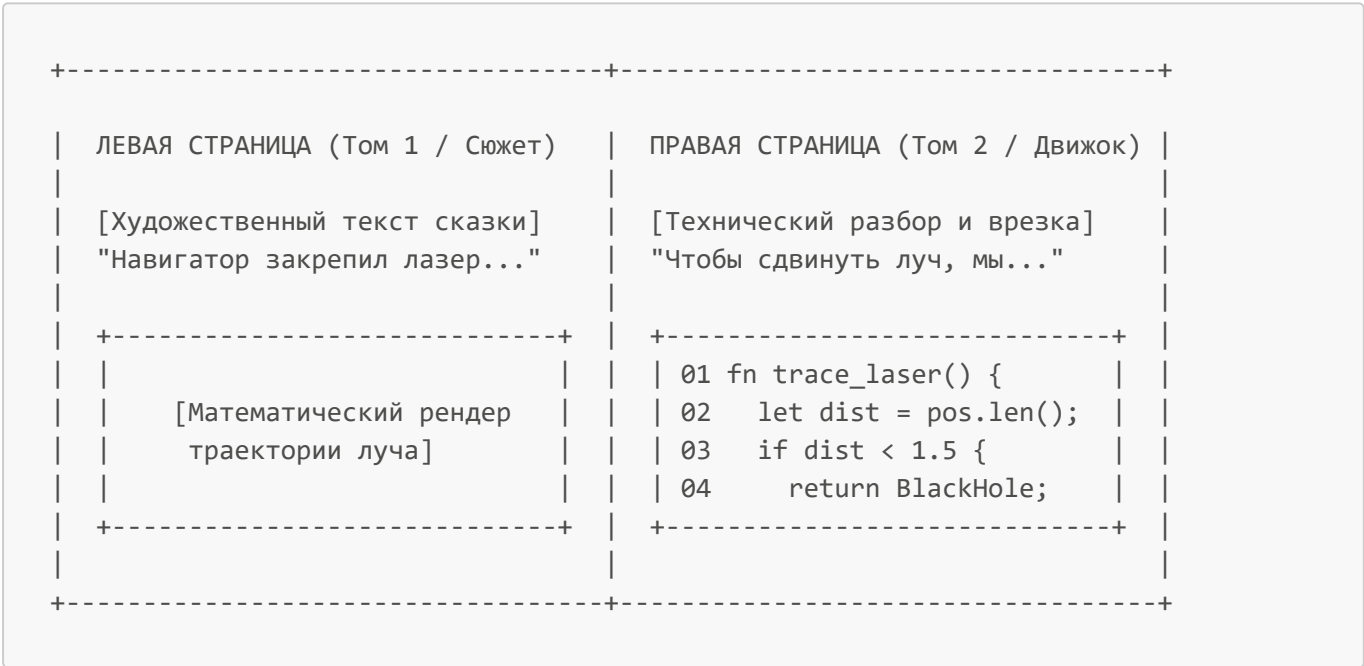
```
}  
}  
}
```

Концепция визуальной верстки разворотов серии «Направление в никуда»

Этот документ содержит техническое задание (ТЗ) для художественного редактора, верстальщика и иллюстратора. Он описывает правила организации пространства на печатной странице дилогии, где художественный текст должен гармонично сочетаться с листингами кода на Rust и математической графикой.

1. Архитектурный принцип «Двухэкранной верстки» (Split-Layout)

Каждая ключевая сцена, связанная с навигацией или пространственными аномалиями, занимает **один полный разворот** (две страницы) и делится на три зоны:



- 1. **Левая страница (Слой Представления / Сюжет):** Посвящена исключительно художественному тексту первого тома. Текст идет крупным, легко читаемым шрифтом с засечками (serif). В нижней или центральной части располагается иллюстрация-рендер физического процесса.
- 2. **Правая страница (Слой Логики / Инженерия):** Представляет собой зеркальный технический разбор этой же сцены для второго тома. Содержит объяснение алгоритма и блоки кода на Rust.

2. Требования к блокам кода (Code Blocks)

Листинги кода на Rust должны оформляться по строгим правилам, чтобы не перегружать восприятие ребенка:

- **Подсветка синтаксиса (Syntax Highlighting):** Обязательна контрастная, но мягкая палитра (рекомендуется тема *Solarized Light* или *GitHub Light* для светлой бумаги). Ключевые слова Rust (`fn`, `struct`, `impl`, `match`, `let mut`) выделяются жирным цветным начертанием.
 - **Нумерация строк:** Каждая строчка кода должна иметь сквозной номер на блеклой левой плашке. Это необходимо, чтобы в тексте руководства можно было делать точечные ссылки (например: «Посмотри на строку 02 — здесь мы измеряем расстояние...»).
 - **Шрифт:** Моноширинный, с четким начертанием символов (рекомендуется *JetBrains Mono*, *Fira Code*), чтобы ребенок не перепутал круглые `()`, фигурные `{}` и квадратные `[]` скобки.
-

3. Требования к иллюстрациям (Математическая графика)

Иллюстрации в серии «Направление в никуда» выступают элементом верификации кода.

- **Векторная строгость:** Изображения должны быть выполнены в четком, минималистичном векторном стиле (псевдо-3D или ортогональные 2D-проекции). Никакой абстрактной «фэнтезийной» рисовки от руки.
 - **Координатная привязка:** На графиках траекторий (например, закручивание лазера в черную дыру) обязательно должны присутствовать тонкие линии координатной сетки, оси X, Y, Z и отметки ключевых точек (точка старта Навигатора, радиусы тора), как на экране осциллографа или в CAD-системах. Это подчеркивает, что картинка — результат математического расчета.
 - **Цветовое кодирование элементов:** Цвета графических элементов на левой странице должны семантически совпадать с комментариями в коде на правой странице (например, если вектор гравитации в коде назван `gravity_pull`, на иллюстрации соответствующая стрелка притяжения должна быть покрашена в тот же цвет).
-

4. Элемент «Триумф» (Консольный блок)

В самом низу правой страницы каждого разворотного модуля размещается стилизованная плашка, имитирующая системный терминал (с темным фоном).

В ней прописывается команда вызова и лог успешного выполнения программы:

```
\$ cargo run --bin chapter_5_blackhole
[LOG] Симуляция запущена...
[LOG] Предупреждение: Луч захвачен гравитационным капканом!
[LOG] Статус вычислений: Триумф!
```

Эта плашка служит визуальным якорем, связывающим печатную страницу книги с реальным монитором компьютера читателя.